arXiv:1904.02441v1 [cs.CR] 4 Apr 2019

Malware Detection using Machine Learning and Deep Learning

Hemant Rathore, Swati Agarwal, Sanjay K. Sahay and Mohit Sewak

BITS, Pilani, Dept. of CS & IS, Goa Campus, Goa, India

Email: hemantr, swatia, ssahay, p20150023@goa.bits-pilani.ac.in

Abstract

Research shows that over the last decade, malware have been growing exponentially, causing substantial financial losses to various organizations. Different anti-malware companies have been proposing solutions to defend attacks from these malware. The velocity, volume, and the complexity of malware are posing new challenges to the anti-malware community. Current state-of-the-art research shows that recently, researchers and anti-virus organizations started applying machine learning and deep learning methods for malware analysis and detection. We have used opcode frequency as a feature vector and applied unsupervised learning in addition to supervised learning for malware classification. The focus of this tutorial is to present our work on detecting malware with (1) various machine learning algorithms and (2) deep learning models. Our results show that the Random Forest outperforms Deep Neural Network with opcode frequency as a feature. Also in feature reduction, Deep Auto-Encoders are overkill for the dataset, and elementary function like Variance Threshold perform better than others. In addition to the proposed methodologies, we will also discuss the additional issues and the unique challenges in the domain, open research problems, limitations, and future directions.

Keywords: Auto-Encoders, Cyber Security, Deep Learning, Machine Learning and Malware Detection.

1 Introduction

In the digital age, malware have impacted a large number of computing devices. The term malware come from **mal**icious soft**ware** which are designed to meet the harmful intent of a malicious attacker. Malware can compromise computers/smart devices, steal confidential information, penetrate networks, and cripple critical infrastructures, etc. These programs include viruses, worms, trojans, spyware, bots, rootkits, ransomware, etc. According to Computer Economics[1], financial loss due to malware attack has grown quadruple from $3.3 billion in 1997 to $13.3 billion in 2006. Every few years the definition of *Year of Mega*

[1] `https://www.computereconomics.com/article.cfm?id=1225`

Breach has to be recalibrated based on attacks performed in that particular year. Recently in 2016, *WannaCry ransomware attack*[2] crippled the computers of more than 150 countries, doing financial damage to different organizations. In 2016, Cybersecurity Ventures[3] estimated the total damage due to malware attacks was $3 trillion in 2015 and is expected to reach $6 trillion by 2021.

Antivirus software (such as Norton, McAfee, Avast, Kaspersky, AVG, Bitdefender, etc.) is a major line of defense for malware attacks. Traditionally, an antivirus software used the signature-based method for malware detection. Signature is a short sequence of bytes which can be used to identify known malware. But the signature-based detection system cannot provide security against zero-day attacks. Also, malware generation toolkits like Zeus [1] can generate thousands of variant of the same malware by using different obfuscation techniques. Signature generation is often a human-driven process which is bound to become infeasible with the current malware growth.

In the past few years, researchers and anti-malware communities have reported using machine learning and deep learning based methods for designing malware analysis and detection system. We surveyed these systems and divided the existing literature into two lines of research. (1) **feature extraction and feature reduction**: In malware analysis, features can be generated in two different ways: static analysis and dynamic analysis. In static analysis, features are extracted without executing the code whereas in dynamic analysis features are derived while running the executable. Ye et al. [17] used Windows API calls obtained from the static analysis as they can reflect true intent or behavior of an attacker. Their experiments show that few API calls like *OpenProcess, CloseHandle, CopyFileA* etc. always co-occur in malicious executables. Raff et al. [20] concluded that byte level n-gram could gather a lot of information about maliciousness from the code section as compared to portable executable header or import sections in a binary file. Strings also contain crucial semantic details, and they can often reflect the attackers real intent and goals. Studies show that in a particular malware family, sample executables often share a similar group of opcodes [16]. Also, few opcodes are more dominant in malicious files as compare to benign executables which can act as a distinguisher. During malware analysis often features vector become extensively large, and it can have a negative impact during modeling. Literature shows various feature selection methods like document frequency [8], information gain [7], max-relevance algorithm [18] have been used in various malware detection systems. Azar [19] performed feature reduction using auto-encoders (in turn reducing the memory requirement) and applied various classification algorithms to achieve higher accuracy. David et al. [2] used a deep stack of de-noising auto-encoders implemented as deep belief network to generate the reduced feature set. (2) **Building Classification Models**: After feature extraction each file can be represented as a feature vector which can be used by the classification algorithm to build a model for malware detection. Firdausi et al. [3] used naive bayes, J48, decision

[2] https://www.cbsnews.com/news/wannacry-ransomware-attacks-wannacry-virus-losses

[3] https://cybersecurityventures.com/hackerpocalypse-cybercrime-report-2016

tree, k-nearest neighbor, multi-level perceptron and support vector machine on features extracted (using dynamic analysis) and achieved the highest accuracy of 96.8% with J48. Moskovitch [8] generated feature vectors with the byte n-gram method and applied feature selection based on document frequency and gain ratio. They reported highest accuracy by selecting top 300 5-gram terms with decision tree and artificial neural network. In 2013, Santos et al. [11] generated a combined feature vector from the static analysis (sequence of opcode frequency) and dynamic features (system call, exception, etc.) from a sample of 1000 malicious and 1000 benign files. Hardy et al. [4] in 2016 used Windows API calls as features with stacked autoencoder for malware detection and achieved an accuracy of 96.85%.

2 Experimental Setup

We formulate the problem of malware analysis and detection as a binary classification problem where malware and benign are the two classes. Figure 1 shows the proposed approach is a multi-step process consisting of various phases performing several tasks: collection of the dataset, disassembling of executable files, feature extraction, dimension reduction, building classification models, and empirical analysis of the results based on different metrics. We discuss each of these phases in the following subsections.

2.1 Dataset

To conduct our experiments, we gathered malware and benign executables from different sources. We downloaded malware samples from an open source repository known as Malicia Project[4]. In Malicia Project, Nappa et al. [9] have collected 11,688 malware samples on Windows platform belonging to a total of 55 different malware families The data collection is performed over a span of 11 months (07/03/2012 to 25/03/2013) from more than 500 drive-by download servers also known as exploit servers. Typically these servers are deployed for a lifetime of 16 hours while some servers even operated for months to spread the malware files. Many malware executables in the dataset will connect to the internet without user consent to perform some cybercrime operation. Most of the malicious executable will also repack themselves on an average of 5.4 times in a day to evade the antivirus signature-based detection system. Thus opcode frequency as a feature can be an excellent measure to detect these malware.

To collect benign executable samples for our dataset, we gathered default files installed in different Windows operating system. VirusTotal[5] is an antivirus aggregator that can be used to check whether an executable is malicious or benign. We declare a sample as non-malicious/benign when all the antivirus from virustotal.com declares it as harmless. We combine the malware and

[4] `http://malicia-project.com/`
[5] `https://www.virustotal.com/`

benign executable files downloaded from different sources (Malicia and Windows) and use it as our experimental dataset. Thus the dataset contains $11,688$ malware and $2,819$ benign executable files.

2.2 Disassembling of Malicious and Benign Executables

As discussed in section 2.1 our data set consist of $14,507$ executable files. To generate the features, we disassemble them by converting an executable file (.exe) to assembly code (.asm). We used object dump utility which is a part of GNU Binutils package[6]. During disassembling few executable files were found to be corrupted or encrypted thus those files were removed from the dataset. Finally, we used $2,819$ benign and $11,308$ malware executables to generate the feature vector and to build the classification model.

2.3 Creation of Feature Vector space

In any machine learning algorithm, the feature vector is a critical component. We generate our feature vector by the static analysis of executable files. In static analysis, discriminatory attributes are collected without the execution of code. Literature shows that various static attribute such as Windows API calls [17] [15], strings [15], opcode [14] [10], and control flow graph [15], etc. are used to separate the malicious and benign executables. We used opcode frequency as a discriminatory feature. Firstly an exhaustive feature list called as *master opcode list* of $1,600$ unique opcodes was created. We future generate a feature vector where rows represent the file name, and columns represent the opcode frequency. Each entry in the vector space represents the number of occurrence of a particular opcode in that file. Finally, the vector space of 2819×1600 for benign and 11308×1600 for malware executables was generated.

2.4 Other issues

Since there is a significant difference between the number of malware ($11,308$) and benign executables ($2,819$) in our dataset, thus it will lead to class imbalance problem. Various methods are available to solve class imbalance problem like random sampling (oversampling/undersampling), cluster-based sampling, ADASYN [5], etc. We used Adaptive Synthetic sampling approach for imbalanced learning (ADASYN) which is an oversampling method for minority class tuples. It synthetically generates data points of minority class based on the k-nearest neighbor algorithm.

As discussed in section 2.3, our dataset contains a large number of features and executable files thus we used cross-validation to generalize our model to an independent dataset. We used 3-fold cross validation in all our experiments. In rotation estimation (a.k.a. cross-validation) data is split into three equal parts where two blocks are used to training the model, and remaining one block is

[6] `https://www.gnu.org/software/binutils/`

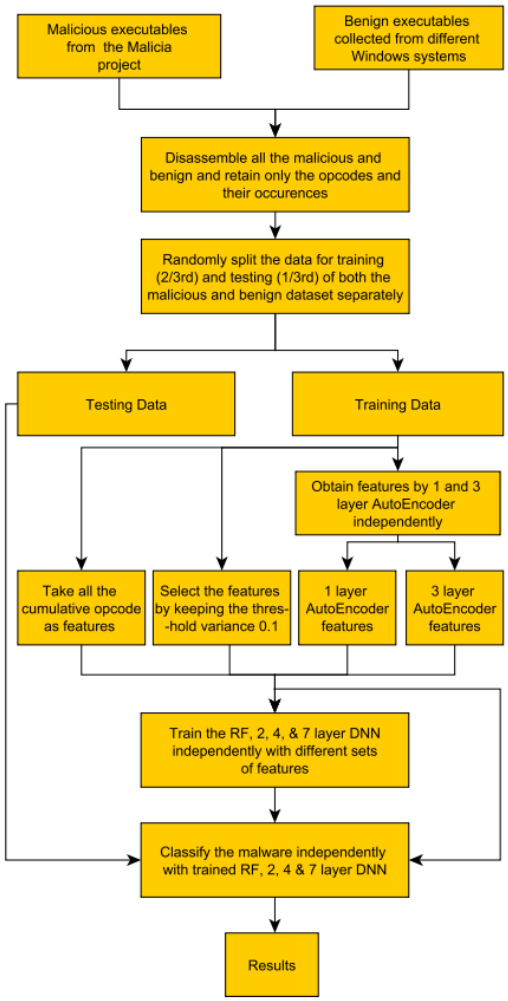


Figure 1: Flowchart for the classification of malware with different sets of features. (Source: Sewak et al. [12])

used for testing. The above exercise is done three times to accommodate all possible combinations.

3 Modelling Malware Detection

As discussed in section 2, malware detection is a binary classification problem. After disassembling the executable samples (malware/benign), successfully generating the feature vectors and using ADASYN, the next steps are dimensionality reduction and then finally building the classification models.

3.1 Dimensionality Reduction

In statistics and machine learning, dimensionality reduction is a process of reducing the number of features under consideration. Our feature vector suffers from the curse of dimensionality since the total number of the unique opcode is $1,600$. When we further analyzed our feature set, we found that for few opcodes the corresponding frequency is zero since the particular opcodes are deprecated. Also for few opcodes, the count was relatively less because they are platform specific and the platform is deprecated. A model created on a dataset suffering from the curse of dimensionality will take a longer time to train and is inefficient in space complexity as well. To choose an optimal number of features we are using different variants of dimensionality reduction methods.

1. *None:* In this method all the opcodes are taken into account for building a classification model without using any feature reduction. We use this as a baseline for different feature reduction methods.

2. *Variance Threshold:* It is a method used to remove the features with low variations. We have removed the attributes with a variance of less than 0.1 assuming they have less prediction power.

3. *Auto-Encoders:* In deep learning auto-encoders are unsupervised learning methods which require only feature vector (opcode frequency), and not class labels for dimensionality reduction.

   (a) A single layer auto-encoder (Non Deep Auto-Encoder), also referred to as AE-1L which contain one encoder layer and a decoder layer.

   (b) A 3-layer stacked auto-encoder(Deep Auto-Encoder), also referred to as AE-3L which contain three encoders followed by three decoders.

   For our experiments, all the auto-encoders use Exponential Linear Unit (ELU) function at all the layers except in the last layer which uses linear

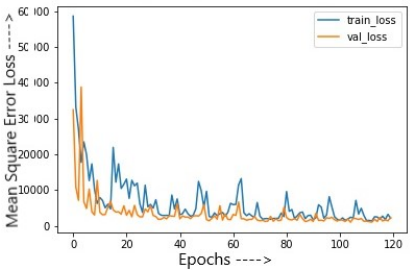


Figure 2: Plot for AE-1L shows mean squared error loss (y-axis) for training and validation across 120 epochs (x-axis) (Source: Sewak et al. [13])

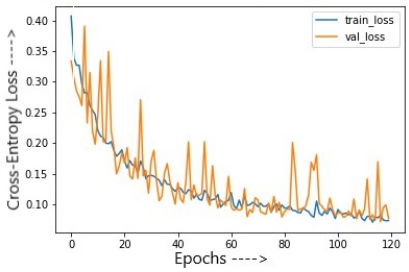


Figure 3: Plot for DNN-2L shows cross entropy loss (y-axis) for training and validation across 120 epochs (x-axis) (Source: Sewak et al. [13])

activation function. In AE-1L, the input directly connects to bottleneck layer which in turn link to the output layer. In both the auto-encoder (AE-1L and AE-3L) models, the bottleneck layer consists of 32 ELU nodes. Thus the architecture of AE-1L is (Input-32-Output) where bottleneck layer will behave as both encoder and decoder. In case of AE-3L where encoder consists of two additional hidden layers connected in sequential order containing 128 and 64 nodes respectively. Similarly, AE-3L decoder comprised of two hidden layers of similar width but connected in reverse order. Thus architecture of AE-3L will be (Input-128-64-32-64-128-Output). For training of both the auto-encoders (AE-1L and AE-3L), the mean square error is used as a loss function over a batch size of 64 samples. Instead of using standard stochastic gradient we have used Adam optimizer [6] to train a batch over 120 epochs. The figure (2) shows the training and validation loss for AE-1L during a complete cycle. The plot shows mean squared error loss (y-axis) for training and validation which are converging around 120 epoch (x-axis).

3.2 Building the learning model

In this paper, we used both machine learning and deep learning based approaches to build the classification models. Based on learning methods we divided our work into two case studies: (1) model based on the Random Forest (RF). In the previous studies [14] [10] conducted on the Malicia dataset [9], we found that tree-based classifier performs better as compared to other classifiers while among tree based classifier RF outperforms others. Thus we finally choose RF from the set of standard classifiers. (2) models based on deep learning.

1. Deep Neural Network using two hidden layers (DNN-2L)
2. Deep Neural Network using four hidden layers (DNN-4L)

3. Deep Neural Network using seven hidden layers (DNN-7L)

We designed multiple models of different depths to learn features at the different level of abstraction. In DNNs all the hidden layers contain ELU activation function except the last. Since malware detection is a binary classification problem, the last layer comprises of softmax activation (sigmoid) function. All the DNNs contain Adam optimizer [6] instead of gradient decent since in general, they have faster convergence rate. Also, we used cross entropy loss function and to avoid overfitting problems we used a dropout rate of 0.1. In DNN-2L, the two hidden layers contain 1024 and 32 nodes respectively. DNN-4L contain four layers with 2^{12-2i} nodes in each layer. Thus DNN-4L hidden layers contains (1024, 256,64,16) nodes. The DNN-7L has seven layers with 2^{11-i} nodes in i^{th} hidden layer. Thus DNN-7L hidden layer contain (1024, 512, 256, 128, 64, 32, 16) nodes. Figure 3 shows the training and validation loss for DNN-2L for a complete cycle of 120 epochs. In this figure, both training and validation loss are gradually decreasing as the model parameters are getting trained in each epoch and finally converged around 120 epoch. Also, something training loss is more than validation loss which is counterintuitive but is it because of the drop-out rate (0.1) during the training cycle.

4 Results

In this section, we will discuss the experimental results obtained after feature reduction (refer section 2.3) with classification models (refer section 3) using various evaluation metrics (accuracy, recall, selectivity, and precision).

Table 1 reveals that for different feature reduction methods we found that VT (combined with RF) based attribute reduction achieved the highest accuracy of 99.78% which is marginally higher than no reduction (None and RF) 99.74% in the feature set. AE-1L performed better than deeper Auto-Encoder (AE-3L) and obtained the highest accuracy (99.41%) with RF. AE-3L based reduction performed lowest in all the methods. Highest True Positive Rate (TPR) of 99.59% was archived by VT (and RF) followed by None, and highest True Negative Rate (TNR) of 100% was achieved by no feature reduction (None and RF).

Table 1 shows that among different classification models, RF outperformed the deep learning models and achieved the highest accuracy of 99.7% (RF and VT). RF again produced the second highest accuracy with no feature reduction. Between different deep learning models, DNN-3L and DNN-7L both combined with AE-1L attained an accuracy of 98.99%. Highest TPR and TNR were produced by RF with VT and None as feature reduction respectively.

5 Conclusion

In the last few years malware have become a significant threat. Classical defense mechanism (like signature-based malware detection) used by anti-virus will fail

Table 1: Results with Features Reduction, Classification Models, Accuracy, Recall /True Positive Rate (TPR), Selectivity /True Negative Rate (TNR), Precision /Positive Predictive Value (PPV) (Source: Sewak et al. [12])

Features	**Classifiers**	**Accuracy**	**TPR**	**TNR**	**PPV**
None	RF	99.74	99.48	100.0	100.0
VT	RF	99.78	99.59	99.97	99.97
AE-1L	RF	99.41	98.86	99.97	99.97
AE-3L	RF	99.36	98.72	100.0	100.0
None	DNN-2L	97.79	96.33	99.26	99.24
VT	DNN-2L	98.84	98.32	99.37	99.37
AE-1L	DNN-2L	96.95	94.57	99.37	99.34
AE-3L	DNN-2L	96.25	93.75	98.79	98.74
None	DNN-4L	97.42	95.38	99.48	99.46
VT	DNN-4L	98.69	97.96	99.42	99.42
AE-1L	DNN-4L	98.99	98.29	99.70	99.70
AE-3L	DNN-4L	97.16	98.61	95.68	95.85
None	DNN-7L	96.15	99.05	93.20	93.66
VT	DNN-7L	96.20	98.89	93.48	93.89
AE-1L	DNN-7L	98.99	98.61	99.81	99.81
AE-3L	DNN-7L	93.60	87.97	99.31	99.23

to cope up new age malware challenges. In this paper, we have modeled malware analysis and detection as machine learning and deep learning problem. We have used best practices in building these models (like cross-validation, fixing class imbalance problem, etc.). We expertly handled the curse of dimensionality by using various feature reduction methods (None, AE-1L and AE-3L). Finally, we compared the models build using RF and DNN (DNN-2L, DNN-4L, and DNN-7L).

Based on our results random forest outperforms all the three deep neural network models in malware detection. We achieved the highest accuracy of 99.78% with random forest and variance threshold which is an improvement of 1.26% on previously reported the best accuracy. Also in feature reduction, variance threshold outplayed auto-encoders in improving the model performance. Another significant contribution of our investigation is a comparison of different combinations of auto-encoder (of depth 1 and 3) and deep neural network (of depth 2, 4 and 7) for malware detection. To our surprise, the best result did not come from any of the deep learning models which indicates that deep leaning may be overkill for Malicia dataset and the trained models are moving towards overfitting.

The same models can be used to detect more complex malware (polymorphic and metamorphic) in the future. Further, it will be interesting to see the effectiveness of other deep learning techniques like recurrent neural network, long short-term memory, etc. for malware detection.

References

References

[1] What is zeus? `https://www.sophos.com/en-us/medialibrary/pdfs/technical%20papers/sophos%20what%20is%20zeus%20tp.pdf`, 2011.

[2] Omid E David and Nathan S Netanyahu. Deepsign: Deep learning for automatic malware signature generation and classification. In *Neural Networks (IJCNN), 2015 International Joint Conference on*, pages 1–8. IEEE, 2015.

[3] Ivan Firdausi, Alva Erwin, Anto Satriyo Nugroho, et al. Analysis of machine learning techniques used in behavior-based malware detection. In *Advances in Computing, Control and Telecommunication Technologies (ACT), 2010 Second International Conference on*, pages 201–203. IEEE, 2010.

[4] William Hardy, Lingwei Chen, Shifu Hou, Yanfang Ye, and Xin Li. Dl4md: A deep learning framework for intelligent malware detection. In *Proceedings of the International Conference on Data Mining (DMIN)*, page 61. The Steering Committee of The World Congress in Computer Science, Computer Engineering and Applied Computing, 2016.

[5] Haibo He, Yang Bai, Edwardo A Garcia, and Shutao Li. Adasyn: Adaptive synthetic sampling approach for imbalanced learning. In *Neural Networks, 2008. IJCNN 2008.(IEEE World Congress on Computational Intelligence). IEEE International Joint Conference on*, pages 1322–1328. IEEE, 2008.

[6] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[7] Mohammad M Masud, Tahseen M Al-Khateeb, Kevin W Hamlen, Jing Gao, Latifur Khan, Jiawei Han, and Bhavani Thuraisingham. Cloud-based malware detection for evolving data streams. *ACM transactions on management information systems (TMIS)*, 2(3):16, 2011.

[8] Robert Moskovitch, Clint Feher, Nir Tzachar, Eugene Berger, Marina Gitelman, Shlomi Dolev, and Yuval Elovici. Unknown malcode detection using opcode representation. In *Intelligence and Security Informatics*, pages 204–215. Springer, 2008.

[9] Antonio Nappa, M Zubair Rafique, and Juan Caballero. Driving in the cloud: An analysis of drive-by download operations and abuse reporting. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*, pages 1–20. Springer, 2013.

[10] Sanjay K Sahay and Ashu Sharma. Grouping the executables to detect malwares with high accuracy. *Procedia Computer Science*, 78:667–674, 2016.

[11] Igor Santos, Felix Brezo, Xabier Ugarte-Pedrero, and Pablo G. Bringas. Opcode sequences as representation of executables for data-mining-based unknown malware detection. *IET Information Sciences*, 231:64–82, 2013.

[12] Mohit Sewak, Sanjay K Sahay, and Hemant Rathore. Comparison of deep learning and the classical machine learning algorithm for the malware detection. In *2018 19th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*, pages 293–296. IEEE, 2018.

[13] Mohit Sewak, Sanjay K Sahay, and Hemant Rathore. An investigation of a deep learning based malware detection system. In *Proceedings of the 13th International Conference on Availability, Reliability and Security*, page 26. ACM, 2018.

[14] Ashu Sharma and Sanjay K Sahay. An effective approach for classification of advanced malware with high accuracy. *International Journal of Security and Its Applications*, 10(4):249–266, 2016.

[15] Yanfang Ye, Tao Li, Donald Adjeroh, and S Sitharama Iyengar. A survey on malware detection using data mining techniques. *ACM Computing Surveys (CSUR)*, 50(3):41, 2017.

[16] Yanfang Ye, Tao Li, Yong Chen, and Qingshan Jiang. Automatic malware categorization using cluster ensemble. In *Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 95–104. ACM, 2010.

[17] Yanfang Ye, Dingding Wang, Tao Li, and Dongyi Ye. Imds: Intelligent malware detection system. In *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 1043–1047. ACM, 2007.

[18] Yanfang Ye, Dingding Wang, Tao Li, Dongyi Ye, and Qingshan Jiang. An intelligent pe-malware detection system based on association mining. *Journal in computer virology*, 4(4):323–334, 2008.

[19] Mahmood Yousefi-Azar, Vijay Varadharajan, Len Hamey, and Uday Tupakula. Autoencoder-based feature learning for cyber security applications. In *Neural Networks (IJCNN), 2017 International Joint Conference on*, pages 3854–3861. IEEE, 2017.

[20] Richard Zak, Edward Raff, and Charles Nicholas. What can n-grams learn for malware detection? In *2017 12th International Conference on Malicious and Unwanted Software (MALWARE)*, pages 109–118. IEEE, 2017.